

BARD Token Audit



June 23, 2025

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Privileged Roles	5
Low Severity	7
L-01 Duplicate Event Emission in mint	7
Notes & Additional Information	7
N-01 Misleading or Inaccurate Comments	7
N-02 Inconsistent Variable Types May Hinder Optimization and Readability	8
N-03 Redundant Zero-Address Check in Constructor	8
Conclusion	10

Summary

Type	DeFi	Total Issues	4 (4 resolved)
Timeline	From 2025-06-06 To 2025-06-17	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	1 (1 resolved)
		Notes & Additional Information	3 (3 resolved)

Scope

OpenZeppelin audited the [Liquid-Bitcoin/BARD](#) repository at commit [9406679](#).

In scope were the following files:

```
contracts
├── BARD
│   ├── BARD.sol
│   └── IBARD.sol
```

System Overview

The `BARD` contract implements an ERC-20-compliant token with extended functionalities, leveraging multiple ERC-20 extensions from the OpenZeppelin library. Specifically, `BARD` inherits from `ERC20Votes`, enabling the on-chain tracking of historical voting power for each account. Each BARD token represents one vote. However, by default, token balances do not count toward voting power. Users must explicitly delegate voting rights to themselves or another account for their voting power to be tracked. For more details on this behavior, refer to the `ERC20Votes` documentation.

The `BARD` contract also inherits the following contracts:

- `ERC20Permit`: Allows token holders to authorize allowances via off-chain signatures, enabling gasless approvals.
- `ERC20Burnable`: Enables holders to burn their own tokens or tokens they have been approved to spend.
- `Ownable2Step`: Introduces a two-step ownership transfer process. The current owner can set a new owner, who must then explicitly accept the role. In addition, the `BARD` contract overrides the `renounceOwnership` function, making it revert on each call, which ensures that the contract always has an owner.

Granting the approval of BARD tokens via off-chain signing or signing a delegation message for voting requires including a nonce in the message and incrementing it after the action is executed. A single `_nonces` mapping is used for both actions. Therefore, the nonce associated with an address will increase after either action is performed.

Privileged Roles

The `owner` address is the only privileged role in the system. It is the only address that can mint new BARD tokens to a specified address with the following constraints:

- **Frequency**: Minting is limited to at most once per year.
- **Cap**: Each minting event cannot increase the total token supply at the time of the minting by more than 10%.

On deployment, an [initial supply of 1 billion BARD tokens](#) will be minted to an address specified by the deployer. The `BARD` contract does not specify how the initial supply or any other minted tokens will be distributed.

Low Severity

L-01 Duplicate Event Emission in `mint`

The `mint` function emits a custom `Mint(to, amount)` event. However, the `ERC20._mint` function that is called also emits a `Transfer(address(0), to, amount)` event, which conveys the same information. As a result, two events with the same content are emitted for a single minting action, which may cause confusion for the users. In addition, the constructor also mints BARD tokens but does not emit a `Mint` event.

Consider removing the redundant `Mint` event to simplify the code and maintain consistency.

Update: Resolved at commits [2159d25](#) and [cb57a21](#).

Notes & Additional Information

N-01 Misleading or Inaccurate Comments

Throughout the codebase, multiple instances of misleading or inaccurate comments were identified:

- A comment in the constructor states:

`first mint not allowed until 1 year after deployment`

However, the constructor will mint 1 billion tokens on contract deployment. Thus, it should be clarified that it is not the first mint, but the next mint after initialization.

- The comment above `mint` states:

`Only callable by the owner once per year and amount must be less than max inflation rate`

This description is slightly inaccurate. The `amount` must be less than the product of the maximum inflation rate and the current total supply, not just the maximum inflation rate.

- The comment above `nonces` states:

Override of the `nonces` function to satisfy both `IERC20Permit` and `Nonces`

However, it should correctly state that it overrides `ERC20Permit`. Furthermore, it would be beneficial for the users to explicitly mention that these nonces are used for both token permits and voting delegation.

Consider updating the comments to accurately reflect the logic and intention of the code.

Update: Resolved at commits [2159d25](#) and [cb57a21](#).

N-02 Inconsistent Variable Types May Hinder Optimization and Readability

The `BARD` contract currently defines constants and variables using integer types such as `uint8` for `MAX_INFLATION`, `uint32` for `MINT_WAIT_PERIOD`, and `uint40` for `lastMintTimestamp`. While these sizes suffice for the values involved, since `MAX_INFLATION` and `MINT_WAIT_PERIOD` are constants, they are embedded in the bytecode and not written into a storage slot, meaning there is no benefit from storage packing. Furthermore, the `lastMintTimestamp` will have to be masked to `uint256` every time the function `mint` is called, which will have a slight extra gas cost.

Consider standardizing by using `uint256` for `MAX_INFLATION` and `MINT_WAIT_PERIOD` for improved clarity, and for `lastMintTimestamp` as well in order to slightly reduce gas usage on every `mint` call.

Update: Resolved at commits [2159d25](#) and [cb57a21](#).

N-03 Redundant Zero-Address Check in Constructor

In the constructor of the `BARD` contract, a `check` is performed to ensure that `_treasury` is not the zero address before minting tokens to it. This check is intended to prevent accidentally not setting it during deployment. However, this validation is redundant, as the `ERC20._mint`

function already includes a check which ensures that the recipient is not `address(0)` and reverts with an appropriate message otherwise.

Consider removing the redundant check from the constructor to simplify the code and make it consistent with the `mint` function, which does not include this extra check.

Update: Resolved at commits [2159d25](#) and [cb57a21](#).

Conclusion

The **BARD** contract implements a standard ERC-20 token with some extra features, leveraging OpenZeppelin's contracts.

The audit did not reveal any security concerns. Overall, the code is well structured and adheres to best practices. The Liquid Bitcoin Foundation team was cooperative and provided all necessary context, enabling a smooth and effective audit process.